

Revisão Integrada

MÓDULO 2 - SEMANA 1 - AULA 1

Agenda

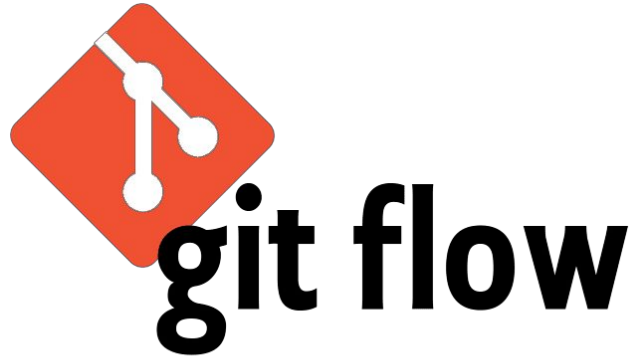
- Fundamentos & Git
- Lógica básica
- Lógica de Controle de Fluxo
- NumPy & Broadcasting
- Pandas & Limpeza

GitFlow

- **Branch 'develop' no GitFlow**

- Qual é a finalidade principal da branch 'develop' e como ela se relaciona com a branch 'main'? De modo geral uma integração, segura, para novas funcionalidades.
 - Finalidade: Servir como ambiente de integração estável para novas funcionalidades.
 - Relação: Recebe merge de feature branches e só vai para a 'main' em releases.

GitFlow



<https://github.com/Desenvolvimento-IA-Analise-Preditiva-T1/Git>

Tipos de Variáveis

- Inteiro (`int`)
- Ponto Flutuante ou Decimal (`float`)
- String (`str`)
- Boolean (`bool`)
- List (`list`)
- Tuple (`tuple`)
- Dictionary (`dict`)

Tipos de Variáveis

```
nome = "Ana"           # Texto (String)
idade = 25              # Número inteiro (Integer)
altura = 1.68           # Número decimal (Float)
tem_carteira = True     # Verdadeiro ou Falso (Boolean)
```

Operadores Aritméticos

Sinal	Nomenclatura	Exemplo
+	Soma	$3 + 2 = 5$
-	Subtração	$3 - 2 = 1$
*	Multiplicação	$3 * 2 = 6$
/	Divisão	$3 / 2 = 1.5$
%	Resto	$3 \% 2 = 1$
**	Exponenciação	$3 ** 2 = 9$
//	Divisão por Inteiro	$3 // 2 = 1$

Álgebra Booleana - not

O comando **not** inverte o sinal de entrada: se a entrada for **Falsa (False)**, a saída será **Verdadeira (True)**; se a entrada for **Verdadeira**, a saída será **Falsa**. Podemos representar esse comportamento através da seguinte tabela verdade:

Entrada (variável)	Saída
False	True
True	False

Álgebra Booleana - and

O operador **and** combina sinais de entrada: se todas as entradas forem **Verdadeiras (True)**, a saída será **Verdadeira (True)**; caso contrário, a saída será **Falsa**. Podemos representar esse comportamento através da seguinte tabela verdade:

Entrada A	Entrada B	Saída
False	False	False
False	True	False
True	False	False
True	True	True

Álgebra Booleana - or

O operador **or** combina sinais de entrada: basta que uma das entradas seja Verdadeira (True) para que a saída seja **Verdadeira**. A saída será **Falsa** somente quando todas as entradas forem **Falsas**. Podemos representar esse comportamento através da seguinte tabela verdade:

Entrada A	Entrada B	Saída
False	False	False
False	True	True
True	False	True
True	True	True

Lógica de Controle de Fluxo

Lógica Booleana: Crucial para mapear decisões (if/elif/else). Use operadores relacionais com cuidado para evitar redundância lógica.

Iteração Controlada: O loop for é ideal para sequências previsíveis, enquanto o while atende a condições dinâmicas de parada.

Sintaxe Limpa: Evite aninhamentos excessivos (nested loops). Prefira modularização e estruturas de decisão diretas.

Tomar Decisões (Estruturas Condicionais)

```
idade = 18

if idade >= 18:
    print("Acesso liberado")
elif idade >= 16:
    print("Acesso permitido apenas com autorização")
else:
    print("Acesso negado")
```

Repetir Tarefas (Laços de Repetição)

```
frutas = ["maçã", "banana", "laranja"]  
  
for fruta in frutas:  
    print(f"Comprar: {fruta}")
```

Repetir Tarefas (Laços de Repetição)

```
contador = 3

while contador > 0:
    print(f"Contagem regressiva: {contador}")
    contador -= 1
```

while: Executa o código enquanto uma condição for verdadeira.

Organizar e Reutilizar (Funções)

```
def calcular_desconto(preco, percentual):  
    desconto = preco * (percentual / 100)  
    return preco - desconto  
  
preco_final = calcular_desconto(100, 15) # 15% de desconto em R$ 100  
print(f"Preço final: R$ {preco_final:.2f}")
```

Permitem agrupar um trecho de código para chamá-lo sempre que precisar, evitando repetições.

Numpy

- O Numpy foi criado para lidar com grandes volumes de dados numéricos de forma eficiente.
- Substitui listas comuns por arrays multidimensionais (ndarrays), que são muito mais rápidos.
- Permite operações vetorizadas, ou seja, aplicar cálculos sobre grandes conjuntos de dados sem precisar de loops.
- É a base de várias bibliotecas como Pandas, Scikit-learn, TensorFlow e Matplotlib.

Estrutura principal do Numpy: ndarray

```
import numpy as np  
lista = [10, 20, 30, 40]  
array = np.array(lista)  
  
print(array)  
print(array.dtype)
```

Criando arrays Numpy

- **A partir de listas:** `np.array([1, 2, 3, 4])`
- **Usando intervalo:** `np.arange(0, 10, 2)`
- **Com espaçamento uniforme:** `np.linspace(0, 1, 5)`
- **Arrays Preenchidos:**
 - `np.zeros(4)`
 - `np.ones(1, 3)`
 - `np.random.rand(3, 3)`

Operações matemáticas e estatísticas

- O Numpy permite aplicar operações vetorizadas diretamente sobre arrays, sem loops:

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40])
```

```
print(arr + 5)
```

```
print(arr * 2)
```

```
print(np.mean(arr))
```

```
print(np.std(arr))
```

Broadcasting

O broadcasting é a capacidade do NumPy de realizar operações aritméticas entre arrays de tamanhos diferentes.

Para funcionar, as dimensões dos arrays (lidas da direita para a esquerda) devem ser compatíveis. Elas são compatíveis apenas se:

- Os tamanhos das dimensões forem exatamente iguais.
- Uma das dimensões for igual a 1.

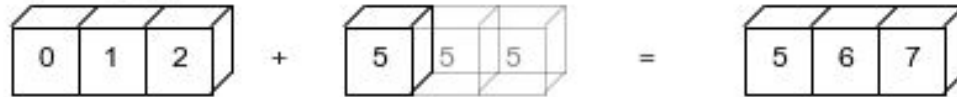
```
A = np.array([10, 20, 30])
```

```
B = 2
```

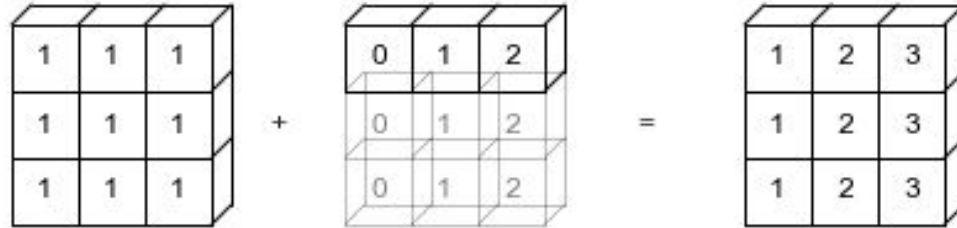
```
resultado = A + B
```

Broadcasting

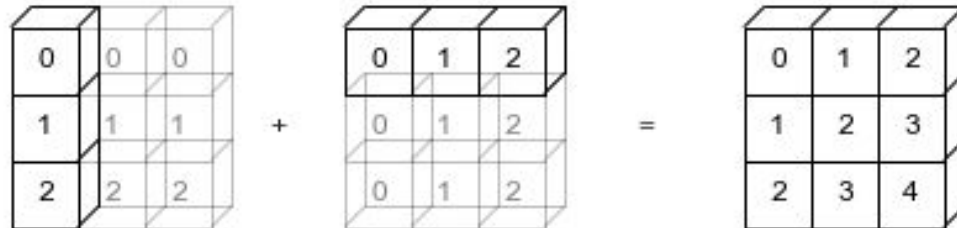
`np.arange(3) + 5`



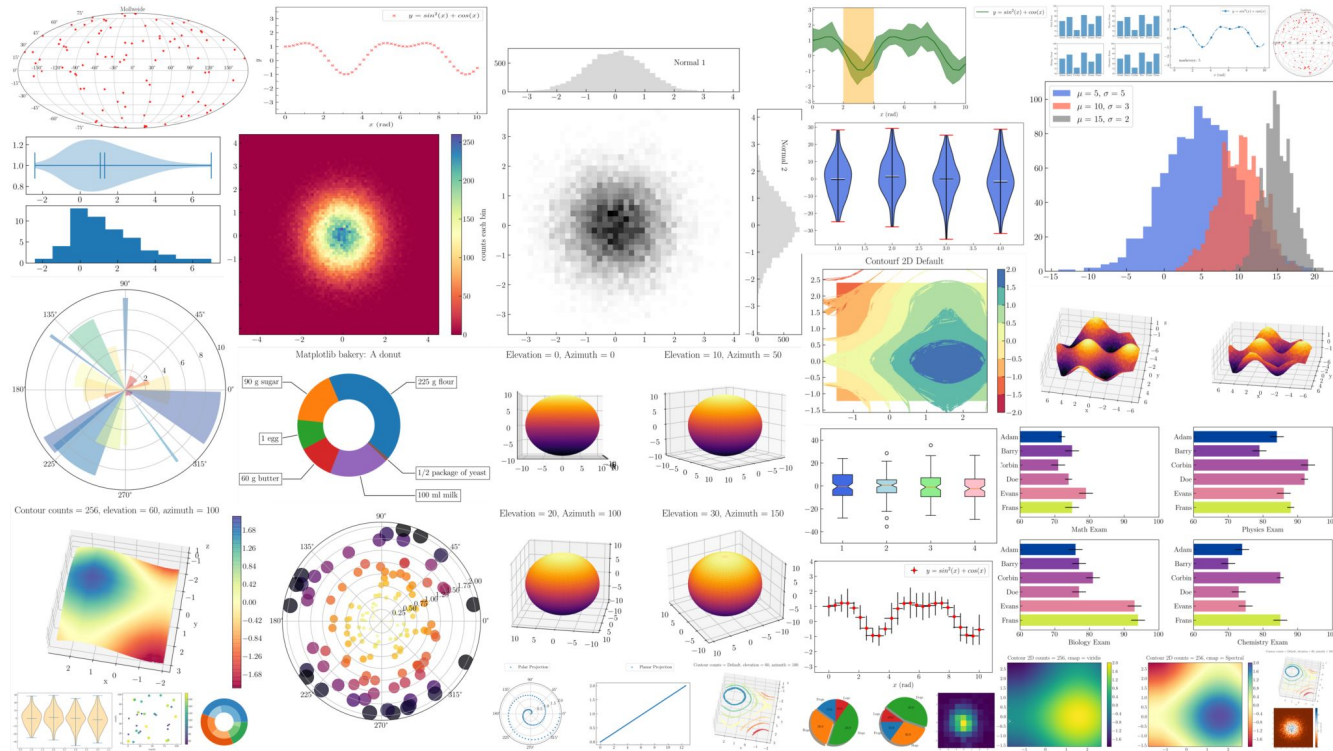
`np.ones((3, 3)) + np.arange(3)`



`np.arange(3).reshape((3, 1)) + np.arange(3)`

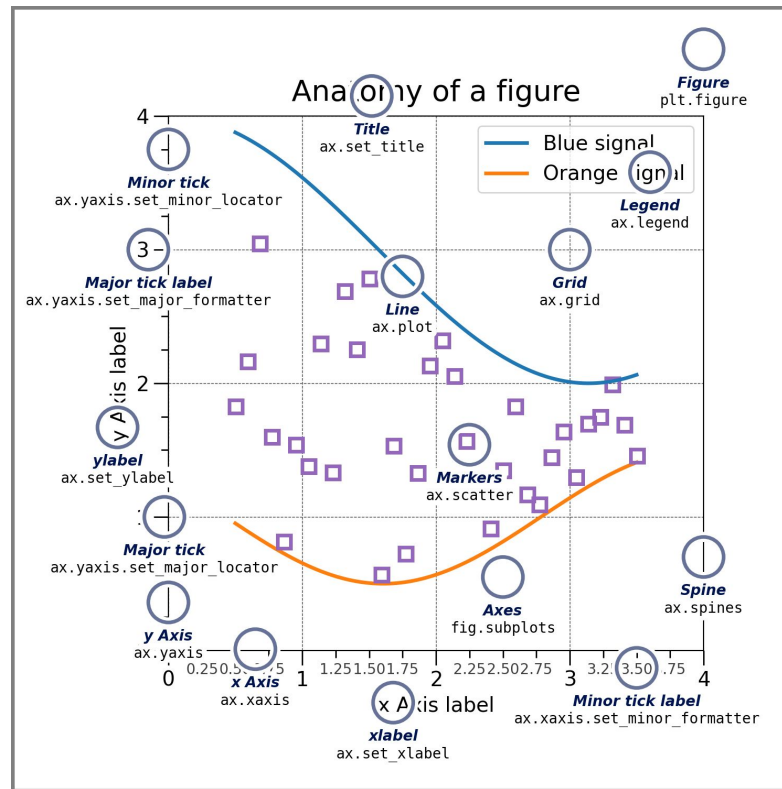


Visualização gráfica



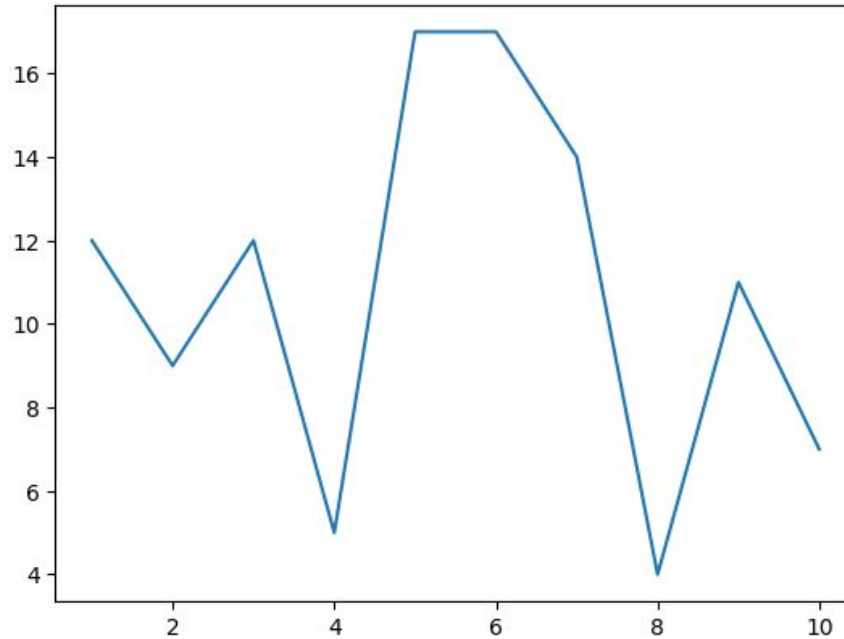
<https://towardsdatascience.com/visualizations-with-matplotlib-part-1-c9651008b6b8/>

Parts of a Figure



https://matplotlib.org/stable/users/explain/quick_start.html

Plot()



```
plt.plot(eixo_x, eixo_y)    # Plot() simples  
plt.show()                 # exibição
```

Plot()

```
# Plotagem personalizada
plt.plot(eixo_x, eixo_y, label='Dados Aleatórios', color='purple', linestyle='--', marker='o')

# Títulos e Legendas
plt.title('Gráfico de Linha Personalizado')
plt.xlabel('Eixo X (Semanas, Dias ou Valores)') # tipo de dado mais usado
plt.ylabel('Eixo Y (Valores / Medidas)')

# Configuração dos eixos (Ticks)
# Ajustado para mostrar de 0 a 10 de dois em dois, cobrindo todos os dados do eixo_x
plt.xticks(range(0, 11, 2))
# Ajustado de 0 a 20 de cinco em cinco, cobrindo a variação máxima do eixo que definimos
plt.yticks(range(0, 21, 5))

# demais config
plt.grid(True, linestyle=':', alpha=0.6) # Adicionado: Linhas de grade de fundo pontilhadas e suaves (alpha)
plt.legend() # Mostra a legenda com o nome definido no 'label' do plt.plot

plt.show()
```

Plot()



Documentação Gráficos

<https://matplotlib.org/stable/gallery/showcase/anatomy.html>

https://matplotlib.org/stable/users/explain/quick_start.html

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html?highlight=pyplot%20plot#matplotlib-py

https://matplotlib.org/stable/api/_as_gen/matplotlib.lines.Line2D.html#matplotlib.lines.Line2D.set_marker

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.scatter.html#matplotlib-pyplot-scatter

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.bar.html#matplotlib-pyplot-bar

https://matplotlib.org/stable/api/figure_api.html?highlight=sharex#module-matplotlib.figure

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.subplot.html

<https://seaborn.pydata.org/>

<https://seaborn.pydata.org/generated/seaborn.scatterplot.html#seaborn.scatterplot>

<https://seaborn.pydata.org/generated/seaborn.relplot.html#seaborn.relplot>

<https://seaborn.pydata.org/generated/seaborn.lineplot.html#seaborn.lineplot>

Pandas

O Pandas é a principal biblioteca do Python para manipulação e análise de dados. Ele transforma dados desestruturados em tabelas organizadas e fáceis de analisar.

Tudo no Pandas gira em torno de duas estruturas:

Series: Uma coluna individual (array 1D rotulado).

DataFrame: A tabela completa com linhas e colunas (2D), parecida com uma planilha do Excel.

Importação e Carga de Dados

```
# Ler arquivos externos
# df = pd.read_csv("dados.csv")
# df = pd.read_excel("dados.xlsx")

# Criar um DataFrame manualmente
dados = {
    "Nome": ["Ana", "Bruno", "Caio", "Diana"],
    "Idade": [25, 30, 22, 28],
    "Setor": ["TI", "Vendas", "TI", "Vendas"],
    "Salario": [5000, 4200, 3800, 4600]
}
df = pd.DataFrame(dados)

# Print dataframe
display(df)
```

	Nome	Idade	Setor	Salario
0	Ana	25	TI	5000
1	Bruno	30	Vendas	4200
2	Caio	22	TI	3800
3	Diana	28	Vendas	4600

Inspeção Rápida (Entendendo o Dataset)

Comando	Para que serve
<code>df.head(n)</code>	Mostra as primeiras <code>n</code> linhas (padrão é 5).
<code>df.tail(n)</code>	Mostra as últimas <code>n</code> linhas.
<code>df.info()</code>	Exibe o tipo de cada coluna, memória usada e contagem de nulos.
<code>df.describe()</code>	Gera estatísticas (média, desvio padrão, min, max) de colunas numéricas.
<code>df.shape</code>	Retorna uma tupla com a quantidade de linhas e colunas: <code>(linhas, colunas)</code> .
<code>df['Setor'].value_counts()</code>	Conta quantas vezes cada valor aparece na coluna.

Seleção e Filtros

```
# Selecionar uma coluna (retorna uma Series)
nomes = df["Nome"]

# Selecionar múltiplas colunas (retorna um DataFrame)
subconjunto = df[["Nome", "Salario"]]

# Filtrar linhas por condição
ti = df[df["Setor"] == "TI"]
salarios_altos = df[(df["Salario"] > 4000) & (df["Setor"] == "Vendas")]

# Busca por posição indexada (.iloc) vs rótulo (.loc)
primeira_linha = df.iloc[0]           # Pela posição numérica (linha 0)
elemento = df.loc[0, "Nome"]          # Pelo índice da linha e nome da coluna
```

Limpeza e Modificação de Dados

```
# Criar uma nova coluna calculada
df["Salario_Anual"] = df["Salario"] * 12

# Tratar valores nulos (ausentes)
df.isna().sum()           # Conta quantos nulos existem por coluna
df_limpo = df.dropna()    # Remove linhas com valores nulos
df["Salario"] = df["Salario"].fillna(0) # Substitui nulos por 0

# Alterar o tipo de dado de uma coluna
df["Idade"] = df["Idade"].astype(float)

# Renomear colunas
df = df.rename(columns={"Setor": "Departamento"})
```

Agrupamento e Agregação

```
# Média de salário por departamento
df.groupby("Departamento")["Salario"].mean()

# Aplicar múltiplas métricas ao mesmo tempo
df.groupby("Departamento").agg({
    "Salario": ["mean", "max"],
    "Idade": "count"
})
```

	Salario		Idade
	mean	max	count
Departamento			
TI	4400.0	5000	2
Vendas	4400.0	4600	2

df					
	Nome	Idade	Departamento	Salario	Salario_Anual
0	Ana	25.0	TI	5000	60000
1	Bruno	30.0	Vendas	4200	50400
2	Caio	22.0	TI	3800	45600
3	Diana	28.0	Vendas	4600	55200

Exportando o Resultado

```
# Salvar o DataFrame em arquivo sem incluir o número do índice do Pandas  
df.to_csv("relatorio_final.csv", index=False)  
# df.to_excel("relatorio_final.xlsx", index=False)
```

Dica de ouro: A maioria das operações no Pandas não altera o DataFrame original diretamente, mas sim retorna uma cópia. Para salvar a alteração, atribua de volta à variável (`df = df.dropna()`) ou use `inplace=True` quando disponível.

ATIVIDADE GUIADA

MACHINE LEARNING E PROGRAMAÇÃO | Atividade Prática de Implementação



Criada com Gemini

Modelos de Machine Learning para Análise Preditiva

MÓDULO 2 - SEMANA 1 - AULA 2

Agenda

- Revisão do Pipeline de Machine Learning
- Tipos de problemas preditivos
- Modelos Lineares vs. Não-Lineares
- Regressão Linear Múltipla
- Regressão Polinomial

Pipeline de Machine Learning

Analogia: "O MasterChef do Machine Learning"

① 1. Coleta de Dados (Ingredientes Brutos)

Garbage In, Garbage Out

Ir à feira/produtor e juntar a matéria-prima (arquivos CSV, bancos SQL, APIs). Se o ingrediente estiver estragado, o prato final será ruim.

Pipeline de Machine Learning

Analogia: "O MasterChef do Machine Learning"

② 2. Limpeza de Dados (Lavar e Descascar)

Tratamento de ruído

Remover sujeiras, descartar partes podres (tratar missings/NaN), retirar duplicatas e ajustar formatos para que os ingredientes possam ser usados.

Pipeline de Machine Learning

Analogia: "O MasterChef do Machine Learning"

③ 3. Engenharia de Features (Cortar e Temperar)

Preparação para o algoritmo

Transformar texto em número (ex: fatiar em cubos), normalizar escalas, criar novas colunas que ajudem no sabor (ex: extrair ano/mês de uma data).

Pipeline de Machine Learning

Analogia: "O MasterChef do Machine Learning"

④ 4. Treinamento do Modelo (O Cozimento no Forno)

Onde a matemática atua

AQUI entra o algoritmo preditivo! O forno (modelo) aprende a temperatura e o tempo corretos observando os dados preparados.

Pipeline de Machine Learning

Analogia: "O MasterChef do Machine Learning"

⑤ 5. Validação (A Degustação do Chef)

Ajuste fino

Provar o prato antes de servir ao cliente. Testar com dados novos que o forno nunca viu para verificar se o ponto está correto.

Pipeline de Machine Learning

Analogia: "O MasterChef do Machine Learning"

⑥ 6. Desdobramento / Deploy (Servir ao Cliente)

Produção real

Colocar a receita automatizada no cardápio (subir o modelo para uma API para rodar na nuvem ou no aplicativo final).

Pipeline de Machine Learning



Pipeline de Pré-Processamento de dados para *machine learning*

Síntese da preparação e transformação da matéria-prima

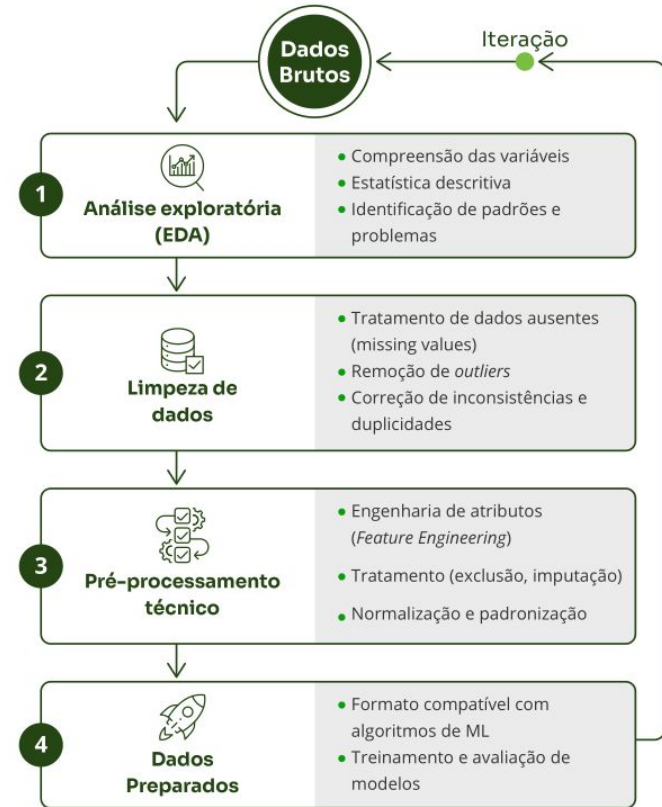
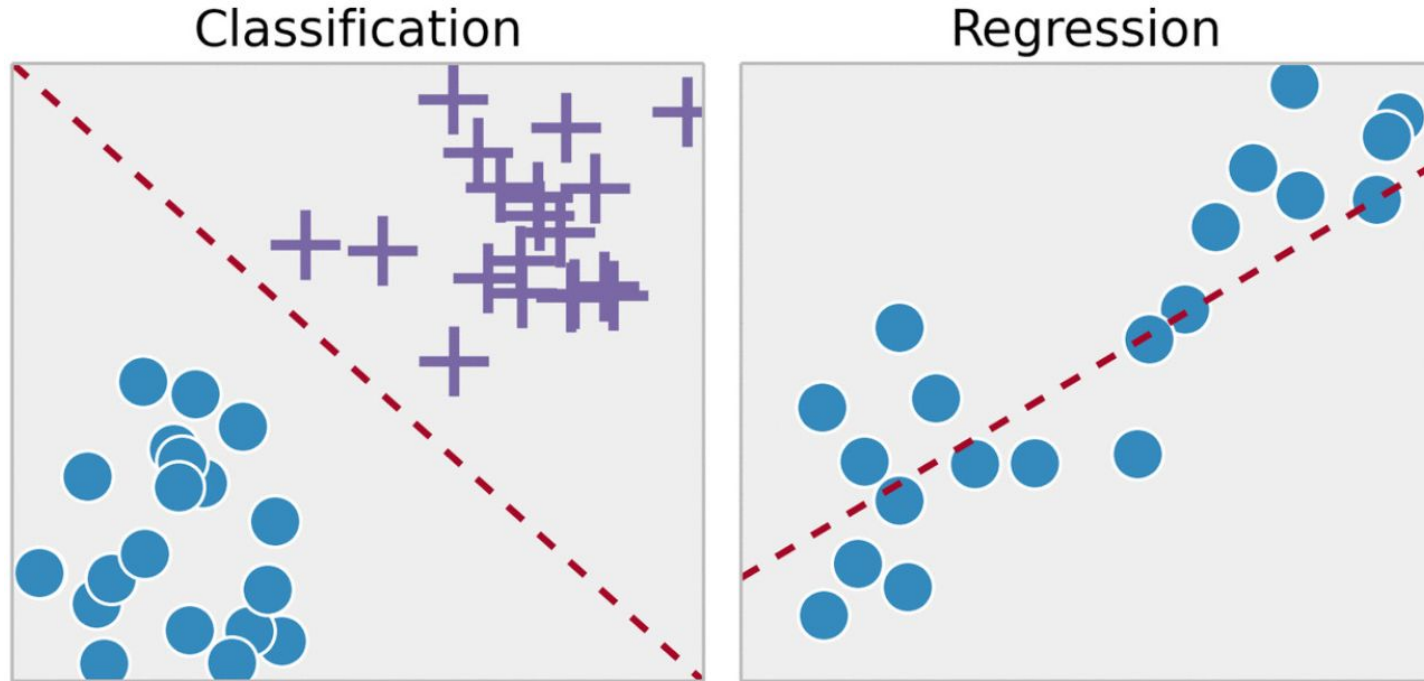


Figura 1 - Pipeline de processamento de dados

Pipeline de Machine Learning

- **Diversos Pipelines de Machine Learning:** Não existe um modelo certo ou errado.
- **A escolha ideal depende de:** Contexto, objetivos e ponto de vista.

Tipos de problemas preditivos (classificação e regressão)



[Shivam Panchal - 2018](#)

Tipos de problemas preditivos (classificação e regressão)

Casos de Regressão:

Precificação Imobiliária: Prever o valor de venda de um imóvel (ex: R\$ 450.500,00) com base na metragem e localização.

Previsão de Demanda (Vendas): Estimar quantos litros de combustível um posto vai vender no próximo final de semana (ex: 12.450 litros).

Tempo de Entrega (Logística): Prever quantos minutos um entregador levará para chegar ao destino (ex: 34,5 minutos).

Tipos de problemas preditivos (classificação e regressão)

Casos de Classificação:

Detecção de Fraude Bancária: Classificar uma transação de cartão de crédito como Legítima ou Suspeita/Fraude.

Diagnóstico Médico: Identificar se uma imagem de raio-X indica um pulmão Saudável ou Com Pneumonia.

Análise de Churn (Cancelamento): Prever se um cliente de telecomunicações vai Cancelar ou Continuar a assinatura no próximo mês.

Definições matemáticas

Matematicamente o Machine Learning busca uma função capaz de relacionar os dados de entrada com os dados de saída:

$$y = f(X)$$

Onde:

y é o alvo (o que queremos descobrir.)

X são as features (o que sabemos)

f é o modelo que explica os dados (função alvo)

Definições matemáticas

Dado esse contexto, em problemas de **aprendizado supervisionado** desejamos identificar a função-alvo f a partir de um conjunto de observações:

$$\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)}) \in \mathcal{X} \times \mathcal{Y} : n = 1, \dots, N\}.$$

Mais especificamente, desejamos encontrar uma função

$$h : \mathcal{X} \rightarrow \mathcal{Y}.$$

Abu-Mostafa et al. (2012)

Definições matemáticas

que seja uma boa aproximação de f Isto é, tal que: $\hat{y} = h(\mathbf{x})$

seja o mais próximo possível de $y = f(\mathbf{x})$ para qualquer par $(\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y}$.

Abu-Mostafa et al. (2012)

Qual é o tipo do seu Target (em resumo)?

Regressão

Target: Valor Numérico Contínuo (float/int na escala numérica).

Pergunta-Chave: "Quanto?" ou "Qual o valor?"

Exemplos: Preço de imóvel, temperatura, tempo de entrega em minutos e previsão de vendas.

Classificação

Target: Classe / Categoria Discreta (rótulos).

Pergunta-Chave: "Qual opção?" ou "Sim/Não?"

Exemplos: Transação fraudulenta (Sim/Não), diagnóstico médico e churn de cliente.

Qual é o tipo do seu Target (em resumo)?

Uma noção que é difundida e vocês vão encontrar em alguns lugares é a de que:

1. **Teste da Casa Decimal:** O resultado pode ter decimais e faz sentido tirar média? É Regressão.
2. **Teste da Etiqueta:** O resultado é uma caixa/rótulo onde você encaixa a resposta? É Classificação.

Exemplo: Prever a temperatura em graus (23.5°C) é Regressão. Prever se o dia estará "Frio" ou "Quente" é Classificação.

Código: Comparando as Duas Tarefas

1. Exemplo de Regressão

```
from sklearn.linear_model import LinearRegression

# Target (y): Preço de Imóveis [R$]
y = [250000.0, 310000.0, 450000.0]

model_reg = LinearRegression()
model_reg.fit(X_train, y_train)

# Predição contínua em R$
preco_pred = model_reg.predict(x_novo)
print(preco_pred) # Ex: [325400.50]
```

2. Exemplo de Classificação

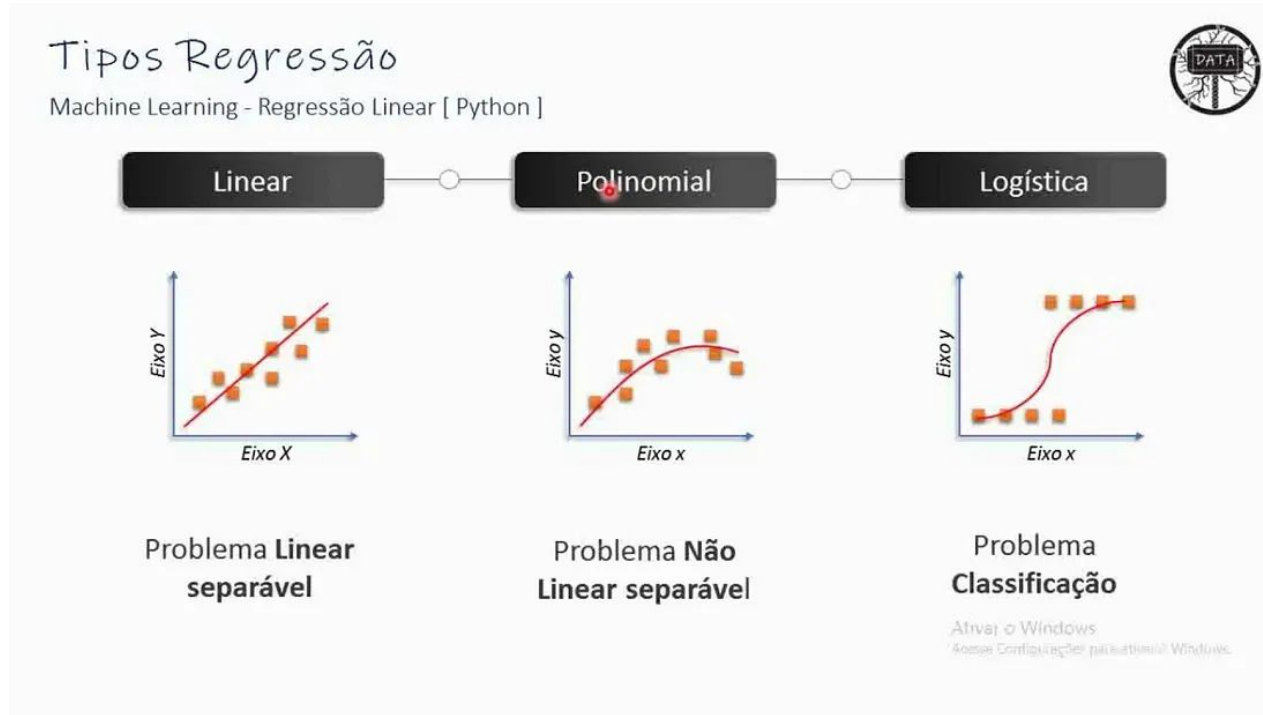
```
from sklearn.linear_model import LogisticRegression

# Target (y): Fraude (1) ou Legítimo (0)
y = [0, 1, 0, 0, 1]

model_clf = LogisticRegression()
model_clf.fit(X_train, y_train)

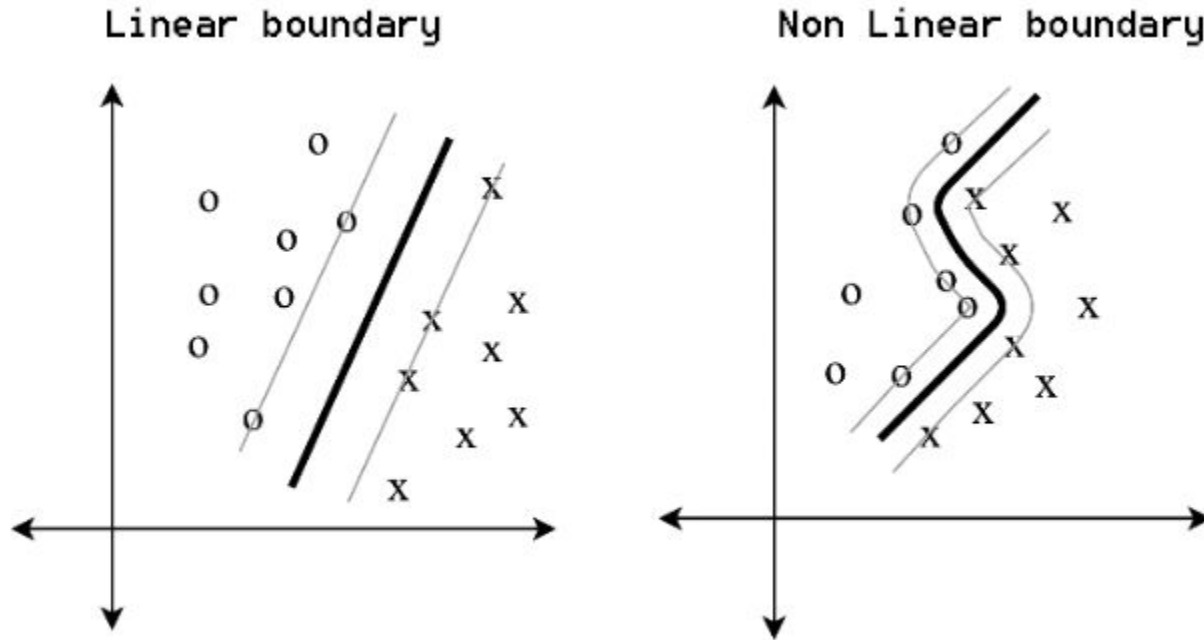
# Predição da classe discreta
classe_pred = model_clf.predict(x_novo)
print(classe_pred) # Ex: [1] (Fraude)
```

Modelos Lineares vs. Não-Lineares



<https://medium.com/@francethais/estudos-de-regress%C3%A3o-1cffc0de8185>

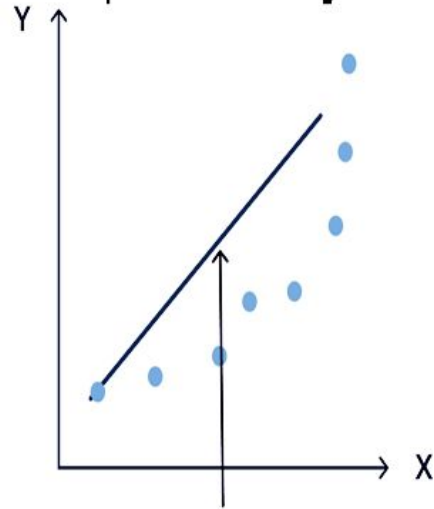
Modelos Lineares vs. Não-Lineares



<https://www.mql5.com/pt/articles/11477>

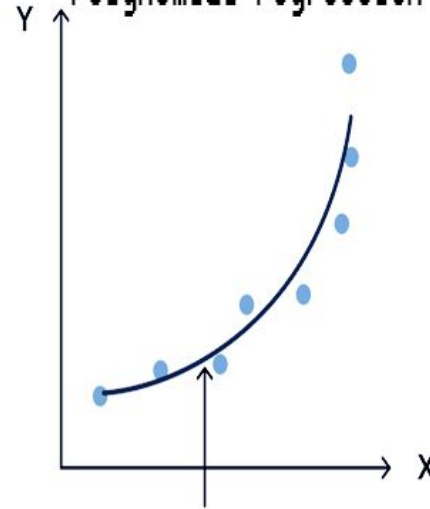
Modelos Lineares vs. Não-Lineares

Simple linear regression



$$y = b_0 + b_1x$$

2nd degree
Polynomial regression



$$y = b_0 + b_1x + b_2x^2$$

<https://www.mql5.com/pt/articles/11477>

Modelos Lineares vs. Não-Lineares

Modelos Lineares

Conceito: Tratam as relações como linhas retas ou planos rígidos.

Algoritmos: Regressão Linear, Regressão Logística.

Vantagens: Altamente interpretáveis, rápidos e leves.

Desvantagens: Não capturam relações complexas ou curvas.

Modelos Não-Lineares

Conceito: Moldam fronteiras flexíveis e regras de decisão em árvore.

Algoritmos: Árvores de Decisão, Random Forest, XGBoost.

Vantagens: Capturam padrões complexos e interações.

Desvantagens: Risco de *overfitting* se não regulados.

Modelos Lineares vs. Não-Lineares

Característica	Modelos Lineares	Modelos Não Lineares
Exemplos de Algoritmos	Regressão Linear, Regressão Logística	Árvores de Decisão, Random Forest, XGBoost
Capacidade de Padrões	Captura apenas tendências simples e diretas.	Captura padrões altamente complexos e interações.
Interpretabilidade	Alta: Fácil explicar a um leigo como cada variável afeta a resposta.	Baixa/Média: Funciona mais como uma "caixa-preta" (exige técnicas adicionais para explicar).
Velocidade / Custo	Extremamente rápidos de treinar e leves.	Mais pesados e exigem maior poder de processamento.
Risco de Overfitting	Baixo (muito rígidos para se moldar ao ruído).	Alto (podem decorar o ruído se não forem regulados).

Regressão Linear Múltipla

Expandindo da reta bidimensional para o hiperplano multidimensional de predição.

Equação Matemática do Modelo

Prevemos uma variável contínua y utilizando a combinação linear de duas ou mais variáveis explicativas ($x_1, x_2, \dots, x_p$).

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$$

β_0 : Intercepto (ponto inicial)

β_i : Coeficientes de inclinação (peso de cada variável)

ε : Erro residual (variação não explicada)

Intuição Geométrica

1 Feature (Simples): Uma reta em um plano 2D.

2 Features: Um plano inclinado em um espaço 3D.

3+ Features: Um **hiperplano** multidimensional.

Interpretação do Coeficiente: Cada β_i mede o impacto isolado de x_i mantendo todas as outras variáveis constantes (*ceteris paribus*).

Regressão Linear Simples

Estrutura: 1 Variável Explicativa (X) $\rightarrow$ 1 Target (y).

Exemplo: Prever o *Preço da Casa* baseado apenas no *Tamanho (m^2)*.

Limitação: Ignora fatores decisivos, resultando em subajuste (*underfitting*) por omissão de variáveis relevantes.

Regressão Linear Múltipla

Estrutura: n Variáveis Explicativas ($X_1, X_2, \dots, X_n$) $\rightarrow$ 1 Target (y).

Exemplo: Prever o *Preço da Casa* usando m^2 , *Nº de Quartos*, *Idade do Imóvel* e *Distância do Centro*.

Vantagem: Explica muito mais a variabilidade do Target e aumenta a precisão do modelo.

Alguns problemas críticos para modelagem

- **Multicolinearidade**

- Duas ou mais variáveis X possuem forte correlação entre si. Causa instabilidade extrema nos coeficientes (β).

- **Heterocedasticidade**

- A variância dos resíduos (erros) não é constante ao longo do Target. Invalida testes de hipóteses estatísticas.

- **Outliers & Pontos Influentes**

- Valores extremos que "puxam" o hiperplano de regressão desproporcionalmente, distorcendo os coeficientes.

Multicolinearidade

O Problema: Ocorre quando duas ou mais variáveis independentes (X) possuem alta correlação entre si (por exemplo: *tamanho da casa em m^2* e *tamanho da casa em pés quadrados*).

Consequência: Os algoritmos têm dificuldade para isolar o efeito individual de cada variável. Os coeficientes (beta) tornam-se extremamente instáveis, variando drasticamente a cada nova amostra.

Diagnóstico: Calculando o **VIF (Variance Inflation Factor)**.

Heterocedasticidade

O Problema: A variância dos erros (resíduos) não é constante ao longo de diferentes faixas de variação do target.

Exemplo: Tentar prever o gasto mensal com lazer baseado na renda. Pessoas de baixa renda variam pouco nos gastos; pessoas de altíssima renda variam enormemente (algumas gastam R\$ 1.000, outras R\$ 50.000).

Consequência: Os p-valores e intervalos de confiança tornam-se não confiáveis.

Regressão Polinomial

Ajustando curvas aos dados mantendo a linearidade nos parâmetros.

Equação do Polinômio

Elevamos as variáveis preditoras a potências de grau d para capturar curvaturas:

$$y = \beta_0 + \beta_1 X + \beta_2 X^2 + \dots + \beta_d X^d + \varepsilon$$

A Pegadinha Teórica: Apesar da curva geométrica, ela continua sendo uma regressão linear nos coeficientes β !

Regressão Polinomial

Por que chamamos de "Regressão Linear"?

1. **Linear nas Variáveis (X): NÃO É LINEAR.** O gráfico no espaço 2D gera uma curva porque temos $X^2, X^3, \dots, X^d$.
2. **Linear nos Parâmetros/Coefficientes (β): É TOTALMENTE LINEAR!**

Repare na estrutura da equação:

$$y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3$$

Regressão Polinomial

Nenhum dos coeficientes β está elevado ao quadrado (β_1^2), multiplicado por outro coeficiente ($\beta_1 \cdot \beta_2$) ou dentro de uma função não-linear (como $\sin(\beta_1)$ ou e^{β_1}).

Se nós fizermos uma simples substituição de variáveis:

- $Z_1 = X$
- $Z_2 = X^2$
- $Z_3 = X^3$

A equação vira:

$$y = \beta_0 + \beta_1 Z_1 + \beta_2 Z_2 + \beta_3 Z_3$$

Regressão Polinomial

Para o computador e para o algoritmo dos Mínimos Quadrados Ordinários (OLS), **isso é exatamente uma Regressão Linear Múltipla simples** rodando em um espaço de novas colunas (Z). Por isso conseguimos resolver o modelo instantaneamente com álgebra matricial sem precisar de algoritmos complexos de otimização não-linear!

Quando utilizar

- 1. Relações Curvilíneas:** Quando o aumento de X acelera ou desacelera a resposta em y (ex: Rendimento agrícola vs. Adubo).
- 2. Picos e Vales:** Padrões sazonais ou de ciclos econômicos que uma reta rígida não consegue acompanhar.

Dilema do Grau



Grau 1 (Reta)

Underfitting (Alto Viés): Reta rígida que ignora a curvatura natural dos dados.



Grau 2 ou 3 (Ideal)

Ajuste Ótimo: Captura a tendência suave sem memorizar ruídos aleatórios.



Grau 15+ (Polinômio Alto)

Overfitting (Alta Variância): Curva oscila insanamente para "tocar" cada ponto do treino.

Dilema do Grau

Explosão de Escala

Se $X = 100$, então $X^3 = 1.000.000$. Elevadas potências alteram drasticamente as ordens de grandeza.

Solução Obrigatória: Aplicar `StandardScaler()` antes de gerar potências!

Perigo nas Pontas (Extrapolação)

Polinômios tendem ao infinito fora do intervalo de treino (Fenômeno de Runge).

Aviso: Nunca use Regressão Polinomial para prever valores muito distantes do intervalo conhecido.

Dilema do Grau

HTML de exemplo

ATIVIDADE GUIADA

MACHINE LEARNING E PROGRAMAÇÃO | Atividade Prática de Implementação



Criada com Gemini

Modelos de Machine Learning para Análise Preditiva

MÓDULO 2 - SEMANA 1 - AULA 3

Agenda

- Regressão Logística
 - Definições e Conceitos

O que é Regressão Logística?

A Regressão Logística é o algoritmo padrão e mais fundamental de Classificação Binária (ou Multiclasse) no Aprendizado de Máquina e na Estatística. Ela calcula a probabilidade de uma observação pertencer a uma determinada categoria (geralmente 1 ou 0, Sucesso ou Fracasso, Sim ou Não).

Alguns exemplos clássicos

Financeiro: O cliente vai pagar o empréstimo (1) ou dar calote (0)?

Medicina: O tumor no exame é maligno (1) ou benigno (0)?

Marketing: O usuário vai clicar no anúncio (1) ou ignorar (0)?

Segurança: A transação de cartão de crédito é fraude (1) ou legítima (0)?

Por que chama Regressão se é Classificação?

- Regressão Interna: O modelo calcula internamente um valor numérico contínuo entre 0.0 e 1.0 (o que é uma probabilidade). Por exemplo:

$$P(Y = 1|X) = 0.83 \text{ (83\% de chance).}$$

- Classificação Externa: Para entregar a resposta final, nós aplicamos uma régua de corte (chamada Threshold ou Limiar de Decisão). Se a probabilidade for maior ou igual a 0.50, classificamos como 1; se for menor, como 0.

Por que chama Regressão se é Classificação?

Resumidamente:

A Regressão Logística estimará uma probabilidade contínua (Regressão) para depois tomar uma decisão discreta (Classificação).

Por que a Regressão Linear Não Funciona para Classificação?

Problema 1: Probabilidades Impossíveis

A reta da Regressão Linear não tem limites horizontais. Se você tentar prever a chance de um cliente dar calote baseado na renda:

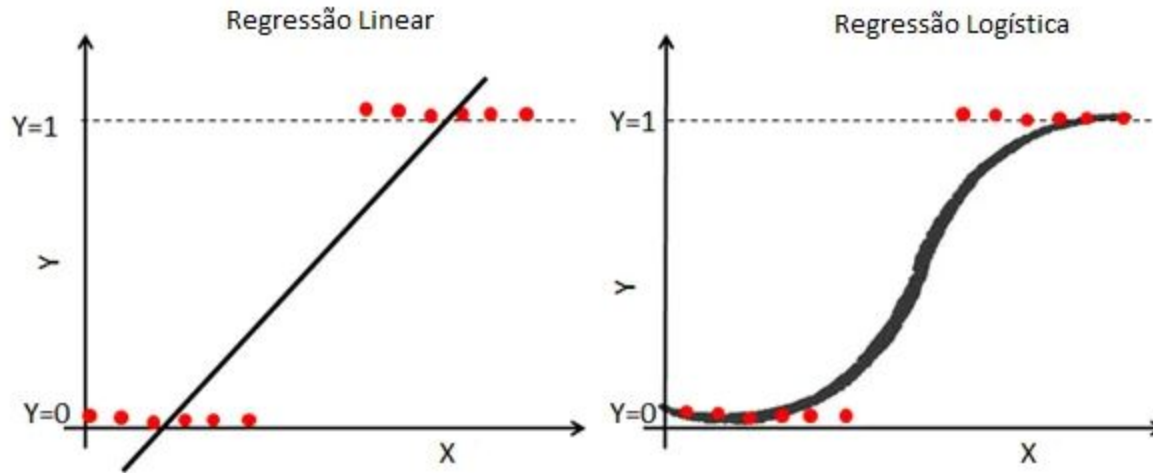
- Para um cliente com renda muito baixa, a reta pode prever $\hat{y} = 1.45$ (145% de probabilidade).
- Para um cliente com renda muito alta, a reta pode prever $\hat{y} = -0.30$ (probabilidade negativa!).
- **Matematicamente:** Probabilidade precisa estar estritamente contida no intervalo $[0, 1]$.

Por que a Regressão Linear Não Funciona para Classificação?

Problema 2: Sensibilidade Extrema a Outliers

Um único ponto distante na base de dados (um outlier) altera drasticamente a inclinação da reta da Regressão Linear, deslocando a linha de corte e fazendo o modelo errar quase todas as classificações anteriores.

Por que a Regressão Linear Não Funciona para Classificação?



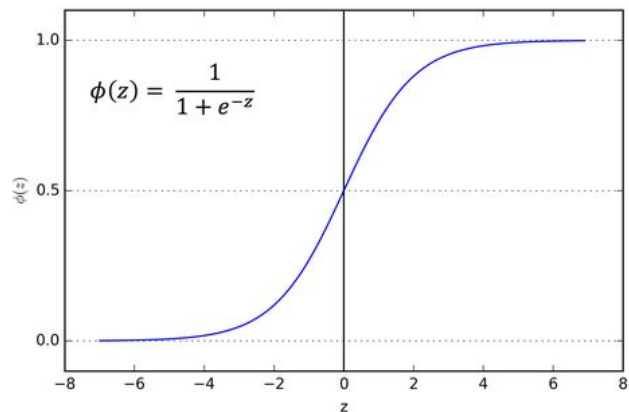
<https://medium.com/@gabstdados/regress%C3%A3o-log%C3%ADstica-%C3%A9-regress%C3%A3o-93e5864ef055>

A Solução: A Função Sigmóide (Logística)

Para resolver a extrapolação de limites, a Regressão Logística pega a combinação linear das variáveis ($z = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$) e a insere dentro de uma função especial em formato de "S", a **Função Sigmóide**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

A Solução: A Função Sigmóide (Logística)



<https://pt.linkedin.com/pulse/understanding-sigmoid-function-logistic-regression-piduguralla?l=pt>

O Comportamento Intuitivo da Sigmóide:

- Quando z é um número positivo muito alto ($z \rightarrow +\infty$), $e^{-z} \rightarrow 0 \implies \sigma(z) \rightarrow 1.0$.
- Quando z é um número negativo muito baixo ($z \rightarrow -\infty$), $e^{-z} \rightarrow +\infty \implies \sigma(z) \rightarrow 0.0$.
- Quando $z = 0$, $\sigma(0) = \frac{1}{1+1} = 0.50$ (50% de incerteza).

AVALIAÇÃO DOCENTE

O que você está achando das minhas aulas neste conteúdo?

Clique aqui ou escaneie o QRCode ao lado para avaliar minha aula.

Sinta-se à vontade para fornecer uma avaliação sempre que achar necessário.

